Ulsan National Institute of Science & Technology

2025-Spring
Intro to Algorithm

2025-06-19

Time allowed: 120 mins (10:30–12:30)
Location: 112-I110

Regulations

1. Please WRITE your student ID of both the quiz paper and the answer sheet. Failure to comply will result in a 20% point deduction.
2. This is a closed-book quiz. No reference materials are allowed.
3. The total score is 92.5
4. Do not tear or damage the paper. Additional materials, including A4 paper, are not permitted.
5. Write all answers on the designated answer sheets in correct question order (e.g., Q1 on the first page, Q2 on the second, etc.). Answers written out of order or misplaced (e.g., Q1 on page 24) will not be graded. If you skip a question, write the question number and leave the space blank.
6. Answers may be written in English or Korean.
7. Use pens instead of pencils. Failure to comply will result in a 10%-point deduction.
8. You must submit the question paper along with your answer sheets. Failure to do so will result in a 20% point deduction.
9. Ensure you have your student ID card. You will be required to present it for identity verification.
10. All electronic devices, including cell phones/pad, must be switched off.
11. You can leave the room after 11:00.
12. The instructor will be available until 11:30. Please review all questions and ask for any necessary clarifications before that time. TAs may not respond to your questions.
13. After 10:45, students cannot join the classroom (criteria: `https://time.is`).
14. It constitutes 40% of the overall grade.
15. Each question will be graded according to the point value specified on the exam paper.
16. For non-simple question, you must provide a clear and detailed explanation to support your answer. Simply writing the final result is not sufficient.
17. For non-simple question, the answer will be evaluated as one of the following: Perfect (10/10) — fully correct with complete and precise explanation; Incomplete (7/10) — largely correct but missing key details or partial justification; Partial (3/10) — many components are missing but shows partial conceptual understanding; Incorrect (0/10) — no valid content provided.

DO NOT OPEN PAPER
UNLESS YOU ARE TOLD TO DO SO

I FULLY UNDERSTAND THE REGULATIONS:

Student ID: _____

Problem 1) Answer the following questions. Note that for each subquestion, your answer should briefly but clearly address each of the required elements. You are encouraged to include comparisons to previous concepts where applicable.

1. Explain the concept of a **binary search**. (Include: motivation in contrast to linear search, core idea, required data structure, time complexity, and limitations.) (4 points)

   Binary search is a search algorithm used on sorted arrays. Unlike linear search, which checks each element one-by-one, binary search repeatedly divides the search interval in half. If the value of the search key is less than the item in the middle, the algorithm continues in the left subarray; otherwise, it continues in the right subarray. It requires a sorted array and operates in $O(\log n)$ time. Its main limitation is that it only works on sorted data and is not suitable for linked lists.

2. Explain the concept of a **binary search tree (BST)**. (Include: motivation in contrast to binary search on arrays, underlying data structure, time complexity analysis, and limitations.)(4 points)

   A Binary Search Tree is a binary tree where each node contains a key, and for each node, all elements in the left subtree are less than the node's key, and all elements in the right subtree are greater. Unlike arrays used in binary search, BSTs allow efficient insertion and deletion in $O(h)$ time, where h is the height. For balanced trees, $h = O(\log n)$, but in the worst case (e.g., skewed tree), $h = O(n)$.

3. Explain the concept of a **balanced binary search tree**. (Include: motivation in contrast to general BSTs, definition and purpose, representative data structure, time complexities, and limitations.) (4 points)

   A balanced BST maintains its height as $O(\log n)$ to ensure efficient operations. This avoids the degenerate case of skewed trees. Examples include AVL trees and Red-Black trees. These trees automatically perform rotations on insertion/deletion to maintain balance. Time complexity for search, insert, and delete is $O(\log n)$. Limitations include more complex implementation and overhead from balancing.

4. Explain the concept of a **splay tree**. (Include: core idea and purpose of splaying, difference from standard BSTs, and use cases.) (2 points)

   A splay tree is a self-adjusting BST where recently accessed elements are moved to the root via rotations (splaying). This brings frequently accessed elements near the top, offering amortised $O(\log n)$ time for operations. Unlike balanced BSTs, splay trees don't maintain strict balance but adapt to access patterns. They are used in caches and access-pattern sensitive scenarios.

Problem 2) The *Set Cover Problem* is defined as follows: Given a finite universe U and a collection of subsets $\mathcal{S} = \{S_1, S_2, \ldots, S_m\}$ such that $\bigcup_{i=1}^{m} S_i = U$, the goal is to find the smallest number of subsets whose union equals U.

1. Prove that the *Set Cover Problem* is in the class NP. (3 points)

   The Set Cover Problem is in NP because given a candidate solution (i.e., a subcollection of $\mathcal{S}$), we can verify in polynomial time whether this subcollection covers all elements of U and whether its cardinality is within the desired bound k. The verification involves checking membership of each $u \in U$ in the union of the chosen subsets, which takes $O(|U||\mathcal{S}|)$ time.

2. Prove that the *Set Cover Problem* is NP-hard. To do so, provide a polynomial-time reduction from the *Vertex Cover* problem, which is known to be NP-complete. (6 points)

   We reduce Vertex Cover to Set Cover. Given an instance of Vertex Cover: a graph $G = (V, E)$ and an integer k, construct an instance of Set Cover as follows. Let the universe U be the set of edges E. For each vertex $v \in V$, define a subset $S_v \subseteq E$ containing all edges incident to v. The collection of subsets is $\mathcal{S} = S_v \mid v \in V$. A vertex cover of size at most k exists in G if and only if there exists a set cover of size at most k in the constructed instance. This reduction is clearly polynomial.

3. Suppose that each element in U appears in at most t subsets in $\mathcal{S}$. Under this condition, the standard greedy algorithm (which iteratively selects the set covering the largest number of uncovered elements per unit cost) achieves what approximation ratio? Justify your answer. (10 points)

   When each element in the universe U appears in at most t subsets in $\mathcal{S}$, the greedy algorithm achieves an approximation ratio of at most t. It holds since the greedy choice ensures that each element is covered at a cost that is at most t times the minimum possible in the optimal cover. This bound is strictly better than the general $\ln n$ bound when t is small. The key idea in the analysis is that no element can be "overcharged" more than t times, as it appears in at most t sets.

Problem 3) The Travelling Salesman Problem (TSP) is a well-known combinatorial optimisation problem. Answer the following subquestions:

1. Prove that the TSP is NP-hard. (Reduce from Hamiltonian Cycle.) (4 points)

   To show that TSP is NP-hard, we reduce from the Hamiltonian Cycle problem, which is known to be NP-complete. Given an undirected graph $G = (V, E)$, construct a complete weighted graph $G' = (V, E')$ where:

   - $w(u, v) = 1$ if $(u, v) \in E$ (i.e., original edge exists),
   - $w(u, v) = 2$ otherwise.

   Then set a cost bound $k = |V|$. There is a Hamiltonian cycle in G if and only if there exists a TSP tour of cost at most k in G'. Thus, solving TSP would solve Hamiltonian Cycle. This shows that TSP is NP-hard.

2. The Held-Karp algorithm is a dynamic programming approach to solve the TSP exactly. Let $C(S, j)$ denote the minimum cost of a path that:

   - Starts at vertex 1,
   - Visits every vertex in subset $S \subseteq \{2, 3, \ldots, n\}$ exactly once,
   - And ends at vertex $j \in S$.

   Write the recurrence relation for $C(S, j)$, where $S \subseteq \{2, \ldots, n\}$, $j \in S$, and $|S| \geq 2$. Also specify the base case(s) and the overall time and space complexity. (6 points)

   Recurrence:

   $$C(S, j) = \min_{i \in S \setminus \{j\}} [C(S \setminus \{j\}, i) + d(i, j)]$$

   Base case:

   $$C(\{j\}, j) = d(1, j) \quad \text{for all } j \in \{2, \ldots, n\}$$

   Final answer:

   $$\min_{j \in \{2, \ldots, n\}} [C(\{2, \ldots, n\}, j) + d(j, 1)]$$

   Time complexity: $O(n^2 2^n)$ **Space complexity:** $O(n 2^n)$

3. Consider the following approximation algorithm for the metric Travelling Salesman Problem (TSP), where the edge weights satisfy the triangle inequality: for all u, v, w, it holds that $d(u, w) \leq d(u, v) + d(v, w)$. The algorithm works as follows:

   - Construct a Minimum Spanning Tree (MST) of the given graph.
   - Traverse the MST in a depth-first manner to obtain a walk that visits all vertices.
   - Convert this walk into a tour by shortcutting repeated vertices.

Explain why this algorithm produces a valid tour, and formally prove that its cost is at most 2 times the cost of an optimal TSP tour. (8 points)

The algorithm produces a valid TSP tour because the MST connects all vertices and the DFS traversal ensures each vertex is visited. When repeated vertices are shortcut (i.e., skipped if already visited), the triangle inequality ensures that the direct connection is no longer than the original path.

Let OPT be the cost of the optimal TSP tour. Let T be the MST. Since an MST is a spanning tree:

$$\text{cost}(T) \leq \text{cost}(\text{OPT})$$

A DFS traversal of the MST gives a walk of cost at most $2 \cdot \text{cost}(T)$. Shortcutting repeated nodes (to form a Hamiltonian cycle) does not increase the total cost due to the triangle inequality. Thus, the cost of the tour satisfies:

$$\text{cost}(\text{tour}) \leq 2 \cdot \text{cost}(T) \leq 2 \cdot \text{cost}(\text{OPT})$$

Approximation ratio: 2.

Problem 4) Kadane's Algorithm for Maximum Subarray Sum

1. Fill in the blanks in the following pseudocode to complete **Kadane's algorithm**, which computes the maximum subarray sum for an array $A[1 \ldots n]$ of real numbers. Write the correct expressions in lines 4, 6, and 8. (1.5 points for each)

```
Function FIND-MAXIMUM-SUBARRAY4(A):
    MaxSum ← 0
    Cur ← 0
    for j ← 1 to n do
        Cur ← _____                          // (line 4)
        if Cur > MaxSum then
            MaxSum ← _____                   // (line 6)
        end
        else if Cur < 0 then
            Cur ← _____                      // (line 8)
        end
    end
    return MaxSum
```

```
Function FIND-MAXIMUM-SUBARRAY4(A):
    MaxSum ← 0; Cur ← 0; for j ← 1 to n do
        Cur ← Cur + A[j]                     // (line 4)
        if Cur > MaxSum then
            MaxSum ← Cur                     // (line 6)
        end
        else if Cur < 0 then
            Cur ← 0                          // (line 8)
        end
    end
    return MaxSum
```

2. What is the time complexity of the above algorithm? Justify your answer. (2 points)

The time complexity is $O(n)$.

Justification: Kadane's algorithm scans the array once using a single loop over n elements. Each iteration performs constant-time operations (max comparison, addition, assignments). No nested loops or recursion is used. Therefore, the total time is linear in the size of the input.

Problem 5) The **0/1 Knapsack Problem** is a classical NP-hard problem in combinatorial optimisation. Answer the following subquestions.

1. The following is the dynamic programming (DP) algorithm for solving the 0/1 Knapsack problem. Complete the missing parts in the pseudocode below.

   Assume that there are n items, each item i having a value $v[i]$ and weight $w[i]$, and that the total knapsack capacity is W. Let $C[i, w]$ denote the maximum value achievable using the first i items with a knapsack capacity w. (10 points)

```
DP-KNAPSACK(n, W)
for w ← 0 to W do
  | C[0, w] ← _____
end
for i ← 1 to n do
  | for w ← 0 to W do
  |   | if w[i] ≤ w then
  |   |   | C[i, w] ← max(_____, _____)
  |   | end
  |   | else
  |   |   | C[i, w] ← _____
  |   | end
  | end
end
return C[n, W]
```

```
DP-KNAPSACK(n, W)
for w' ← 0 to W do
  | C[0, w'] ← 0                                          // (initialise with 0)
end
for i ← 1 to n do
  | for w' ← 0 to W do
  |   | if w[i] ≤ w' then
  |   |   | C[i, w'] ← max(C[i-1, w'], v[i] + C[i-1, w' - w[i]])  // (include or exclude item i)
  |   | end
  |   | else
  |   |   | C[i, w'] ← C[i-1, w']                          // (item i too heavy)
  |   | end
  | end
end
return C[n, W]
```

2. What is the time complexity of the above algorithm? is it polynomial? You must provide a proof of your answer (5 points)

   The DP table has size $(n + 1) \times (W + 1)$, and each entry is filled in constant time. Hence, time complexity is $O(nW)$ and space complexity is also $O(nW)$.

   Although this is polynomial in n and W, the problem is still considered NP-hard because W is not polynomial in the input size — it can be exponential in the number of bits required to represent it. This is known as a pseudo-polynomial time algorithm. So, yes — the time complexity is polynomial in numeric value of W, but not in the size of the input (which depends on $\log W$).

3. The 0/1 Knapsack problem is NP-hard. However, approximation algorithms can give near-optimal solutions in polynomial time. Consider the following approach: You group items based on their value-to-weight ratio and greedily choose items with the highest ratio until the knapsack is full.

(a) Explain whether this greedy algorithm always produces the optimal solution. Provide a counterexample if not (4 points)

No, it does not always produce the optimal solution.

Counterexample: Let $W = 50$ and two items:

- Item 1: $v = 60$, $w = 10 \rightarrow v/w = 6.0$
- Item 2: $v = 100$, $w = 50 \rightarrow v/w = 2.0$

The greedy algorithm chooses item 1. Thus, the total value 60. However, the optimal solution is to take item 2 with total value 100. Thus, greedy fails.

(b) The greedy algorithm described above does not always return the optimal solution. One proposed improvement is to compute two solutions: (8 points)

- One using the greedy strategy (sorting by value-to-weight ratio),
- The other by selecting only the single item with the highest value v_i that fits in the knapsack.

The algorithm then returns the better of the two. Does this modification improve the theoretical worst-case approximation ratio compared to the original greedy algorithm? Prove your answer.

Answer: Yes, this modification improves the theoretical worst-case approximation ratio.

Proof (2-approximation guarantee):

Let **OPT** denote the optimal solution's value for a given instance. Consider two cases:

- **Case 1: Optimal solution contains exactly one item.**
  In this case, selecting the single highest-value item that fits exactly matches the optimal solution, yielding **OPT**.
- **Case 2: Optimal solution contains multiple items.**
  Let the highest-value item in the optimal solution have value $v_{\max}$. Clearly, we have: $v_{max} \geq OPT/2$
  This inequality holds because in the worst case, the largest item accounts for at least half the optimal solution's total value when multiple items are present.
  Since our modified algorithm explicitly evaluates and considers selecting this largest-value item alone, the value obtained by the algorithm satisfies:

$$\text{Value of modified algorithm} \geq v_{max} \geq OPT/2$$

  Thus, the modified algorithm provides at least half of the optimal value, achieving a 2-approximation guarantee.

In contrast, the original greedy algorithm alone does not have a constant-factor approximation guarantee and can perform arbitrarily poorly. Hence, this modification strictly improves the theoretical approximation ratio.

Problem 6) Miscellaneous short-answer problems. (4 points for each)

1. Suppose we construct a decision tree for a comparison-based sorting algorithm, and its height is h. What is the maximum number of leaves the tree can have in terms of h? Also, given n distinct elements, how many distinct comparison paths exist?

   A binary decision tree of height h can have at most 2^h leaves. For sorting n distinct elements, there are $n!$ possible total orderings, so the decision tree must have at least $n!$ leaves. Hence, the minimum height h of such a tree satisfies $2^h \geq n!$, which implies $h \geq \log_2(n!) = \Omega(n \log n)$. Therefore, there are $n!$ distinct comparison paths for sorting n elements.

2. Define the little-o notation (o-notation) formally in the context of asymptotic analysis.

   A function $f(n)$ is $o(g(n))$ if for every constant $\varepsilon > 0$ there exists n_0 such that $|f(n)| \leq \varepsilon |g(n)|$ for all $n \geq n_0$, i.e. $\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$.

3. Briefly describe the Timsort algorithm. What design principle motivates it? What are its core steps, and what is its time complexity in the best and worst case?

   Timsort is a hybrid sorting algorithm used in Python and Java, combining merge sort and insertion sort. It is motivated by exploiting runs—already sorted sequences—in real-world data. Core steps:

   - Identify ascending runs in the array.
   - Use insertion sort to extend small runs.
   - Merge runs using a modified merge sort with stack-based run management.

   Best case: $O(n)$ (nearly sorted data). Worst case: $O(n \log n)$.

4. Explain why Dijkstra's algorithm does not allow negative edge weights. Provide a small counterexample showing incorrect behaviour.

   Dijkstra's algorithm assumes that once a node is visited with the shortest path, no shorter path will be found later. This assumption breaks with negative weights.

   Example: Graph: $A \xrightarrow{1} B$, $A \xrightarrow{4} C$, $C \xrightarrow{-5} B$

   Dijkstra starting from A:

   - Start at A.
   - Visit B via $A \to B$ with cost $1 \to B$ is finalised.
   - Visit C via $A \to C$ with cost 4.
   - Edge $C \to B$ has cost -5, so $4 + (-5) = -1$.
   - But B was already finalised with cost 1, so Dijkstra ignores this shorter path.

   Thus, Dijkstra returns a path cost of 1 for $A \to B$, while the correct shortest path is $A \to C \to B$ with total cost -1. This error occurs because Dijkstra assumes that once the shortest path to a node is found, it cannot be improved — which is invalid when negative weights are present.